CSCI 432/532, Spring 2026

Homework 6

Due Monday, March 2 at 8:30am

Submission Requirements

- Type or clearly hand-write your solutions into a PDF format so that they are legible and professional. Submit your PDF on Canvas.
- Put each **sub**problem on a separate page and select the corresponding problem via the Gradescope interface.
- Do not submit your first draft. Type or clearly re-write your solutions for your final submission. If your submission is not legible, we will ask you to resubmit.

Collaboration and Resources

The *best* resources for completing the homework are (in no particular order):

- Your own notes from lectures and problem sessions, and/or lecture recordings.
- The lecture notes linked from the course website for the lecture day.
- The questions channel on Discord.
- Office hours.
- Discussions with classmates.

You may also access almost any resource in preparing your solution to the homework. The exception is that you may not seek answers to the problems on the homework directly, such as by pasting them into a GenAI tool, searching for solutions on a search engine, looking for them on Chegg or similar websites, or asking another person for the solution.

Additionally, you ***must***

- Write your solutions in your own words—this means that you should never be *copying* anything of substance from any source, and
- credit every resource you use, including GenAI tools, by providing links or full chat transcripts. If you use the provided LaTeX template, you can use the `Sources` environment for this. Ask if you need help!

> Remember, if you choose to use GenAI tools, not only must you provide a full transcript of your conversation (or a link to one), you must also use the following prompt (or some variation of it), and provide a full transcript of the conversation.

Prompt:

You are acting as a teaching assistant for an advanced undergraduate/graduate algorithms and computer science theory course.
Your role is not to provide full solutions or final answers.
Instead, you should act like we are having a conversation. Ask me a single question and let me respond. Avoid asking me many questions at once or giving me a lot of information at one time. Guide me using hints, leading questions, partial insights, and sanity checks. Help me debug my own proofs, algorithms, or reductions. Do not give a complete solution or full proofs. Be patient with me. Don’t give me details so that I can move on to the next part of a problem. Make sure that I understood before moving on.
The course uses Jeff Erickson’s Models of Computation lecture notes, so you should guide me to answer problems using the definitions, notation, and style from those notes.
Assume I am a serious student trying to learn, not to shortcut the assignment. I am attaching the assignment, so you should refuse to provide a complete solution to the problems listed. Of course, you can walk me through the "Solved Problems" listed at the end if I ask.

1. Describe (as in the problem session solutions—note that you do *not* need to give a full specification of states, transitions, etc.) a Turing machine that decides the following language: $\{ww : w \in \{0,1\}^*\}$.

TM rubric. 10 points =

+ 2 for an English description of a Turing machine, regardless of correctness.
+ 6 for correctness.
+ 2 for an appropriate mix of detail and high-level intuition.

2. A ***Turing machine with stay put instead of left*** is similar to an ordinary Turing machine, but the transition function has the form $\delta : Q \times \Gamma \to Q \times \Gamma \times \{R, S\}$. At each point, the machine can move its head right or let it stay put in the same position.

   (a) Show that this Turing machine variant is *not* equivalent to a standard Turing machine.

   (b) What class of languages do these machines recognize?

TM simulation rubric. 10 points =

+ 2 for an English description of either (1) how to simulate a standard Turing machine with the new type of Turing machine or (2) why standard Turing machine can't be simulated with the new type of Turing machine.
+ 6 for correctness.
+ 2 for an appropriate mix of detail and high-level intuition.

3. Consider the language $\textsc{NeverReject} = \{\langle M \rangle : \textsc{Reject}(M) = \emptyset\}$; that is, the set of all (encodings of) Turing machines that do not reject any input.

   (a) Prove that $\textsc{NeverReject}$ is undecidable via ***diagonalization***.

Diagnoalization rubric. 10 points =

+ 4 for a correct wrapper Turing machine
+ 6 for self-contradiction proof (= 3 for $\Longleftarrow$ + 3 for $\Longrightarrow$)

   (b) Prove that $\textsc{NeverReject}$ is undecidable via ***reduction*** (for example, from the halting problem).

Reduction rubric. 10 points =

+ 4 for a correct reduction
+ 3 for "if" proof
+ 3 for "only if" proof

Solved problems

1. A ***Turing machine with a doubly infinite tape*** is similar to an ordinary Turing machine, but the tape is infinite in both directions. Show that this Turing machine variant is equivalent to a standard Turing machine.

Solution: To show that a Turing machine with a doubly infinite tape is equivalent to a standard Turing machine, we can show that a standard Turing machine can simulate a Turing machine with a doubly infinite tape. To do so, we must simulate being able to write arbitrarily far to the left of the starting point. In class, we saw that this can be done by simulating a second tape representing negative indices using our tape alphabet.

Another way to do this is to use the original tape alphabet, but copy the entire tape contents one position to the right whenever we need to write to the left of the starting point. We can add a left sentinel symbol and right sentinel symbol to the tape alphabet to mark the left and right ends of the tape contents. Whenever we need to write to the right sentinel, we shift it one to the right and write the new symbol in its old place. Whenever we need to write to the left sentinel, we shift all the tape contents one to the right and write the new symbol in the left sentinel's old place. This way, we can simulate a doubly infinite tape using a standard Turing machine. ■

2. Conside the language $\textsc{SelfHalt} = \{\langle M\rangle : M \text{ halts on input } \langle M\rangle\}$; that is, the set of all (encodings of) Turing machines that halt when given their own encoding as input. Prove that $\textsc{SelfHalt}$ is undecidable.

Solution: Suppose to the contrary that there is a Turing machine SH such that $\textsc{Accept}(SH) = \textsc{SelfHalt}$ and $\textsc{Diverge}(SH) = \emptyset$.

Let SH^X be the Turing machine obtained from SH by redirecting every transition to `accept` to a new hanging state `hang`, and then redirecting every transition to `reject` to `accept`. Then $\textsc{Accept}(SH^X) = \Sigma^* \setminus \textsc{SelfHalt}$ and $\textsc{Reject}(SH^X) = \emptyset$. It follows that SH^X accepts $\langle SH^X\rangle$ if and only if SH^X does not halt on $\langle SH^X\rangle$, and we have a contradiction. ■

3. Consider the language $\textsc{SometimesHalt} = \{\langle M\rangle \mid M \text{ halts on at least one input string}\}$. Note that $\langle M\rangle \in \textsc{SometimesHalt}$ does not imply that M *accepts* any strings; it is enough that M *halts* on (and possibly rejects) some string.

Prove that $\textsc{SometimesHalt}$ is undecidable.

Solution: We can reduce the standard halting problem to $\textsc{SometimesHalt}$ as follows:

```
DecideHalt(⟨M, w⟩):
  Encode the following Turing machine M′:
      M′(x):
        (ignore x)
        run M on input w
  return DecideSometimesHalt(⟨M′⟩)
```

We prove this reduction correct as follows:

$\Longrightarrow$ Suppose M halts on input w.
Then M' halts on *every* input string x.
So DecideSometimesHalt must accept the encoding $\langle M' \rangle$.
We conclude that DecideHalt correctly accepts the encoding $\langle M, w \rangle$.

$\Longleftarrow$ Suppose M does not halt on input w.
Then M' diverges on *every* input string x.
So DecideSometimesHalt must reject the encoding $\langle M' \rangle$.
We conclude that DecideHalt correctly rejects the encoding $\langle M, w \rangle$.

■